



Smart Contract Security Audit Report



Table Of Contents

1 Executive Summary	_____
2 Audit Methodology	_____
3 Project Overview	_____
3.1 Project Introduction	_____
3.2 Vulnerability Information	_____
4 Code Overview	_____
4.1 Contracts Description	_____
4.2 Visibility Description	_____
4.3 Vulnerability Summary	_____
5 Audit Result	_____
6 Statement	_____

1 Executive Summary

On 2024.07.11, the SlowMist security team received the team's security audit application for Lazyotter Iterative Contracts, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

Serial Number	Audit Class	Audit Subclass
1	Overflow Audit	-
2	Reentrancy Attack Audit	-
3	Replay Attack Audit	-
4	Flashloan Attack Audit	-
5	Race Conditions Audit	Reordering Attack Audit
6	Permission Vulnerability Audit	Access Control Audit
		Excessive Authority Audit
7	Security Design Audit	External Module Safe Use Audit
		Compiler Version Security Audit
		Hard-coded Address Security Audit
		Fallback Function Safe Use Audit
		Show Coding Security Audit
		Function Return Value Security Audit
		External Call Function Security Audit

Serial Number	Audit Class	Audit Subclass
7	Security Design Audit	Block data Dependence Security Audit
		tx.origin Authentication Security Audit
8	Denial of Service Audit	-
9	Gas Optimization Audit	-
10	Design Logic Audit	-
11	Variable Coverage Vulnerability Audit	-
12	"False Top-up" Vulnerability Audit	-
13	Scoping and Declarations Audit	-
14	Malicious Event Log Audit	-
15	Arithmetic Accuracy Deviation Audit	-
16	Uninitialized Storage Pointer Audit	-

3 Project Overview

3.1 Project Introduction

AmbientVault is Lazyotter Iterative project that contains the vault and helper section, The AmbientVaultHelper contract is designed to assist users in interacting with crocSwapDex liquidity pools and AmbientVault. Users can utilize this contract to deposit and withdraw with the crocSwapDex liquidity pools' liquidity tokens to mint shares in the AmbientVault, and swap tokens with the liquidity token with efficient token operations.

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Potential Sandwich Attack Risk	Design Logic Audit	Medium	Fixed
N2	Repeated assignment of the same input	Design Logic Audit	Medium	Fixed
N3	Receive can lock users' native tokens	Others	Low	Acknowledged
N4	Reentrancy reminder	Reentrancy Vulnerability	Suggestion	Fixed
N5	Low-level call reminder	Others	Suggestion	Fixed
N6	Missing the 0 address Check	Others	Suggestion	Fixed
N7	Token compatibility reminder	Design Logic Audit	Suggestion	Acknowledged
N8	External call reminder	Unsafe External Call Audit	Suggestion	Acknowledged

4 Code Overview

4.1 Contracts Description

Audit Version:

<https://github.com/lazyotter-finance/lazyotter-contract>

commit: ee67e53ef6f1506100059153b9ce05df6f9b32d4

Code scope:

/src/vaults/AmbientVault.sol

/src/helper/AmbientVaultHelper.sol

/utils/CrocLpErc20.sol

/utils/FixedPoint.sol

/utils/Zap.sol

Fixed Version:

<https://github.com/lazyotter-finance/lazyotter-contract>

commit: 7afc866fe4bfabc31fab108637e982607208e60d

Code scope:

/src/vaults/AmbientVault.sol

/src/helper/AmbientVaultHelper.sol

/utils/CrocLpErc20.sol

/utils/FixedPoint.sol

/utils/Zap.sol

The main network address of the contract is as follows:

The code was not deployed to the mainnet.

4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

AmbientVaultHelper			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
deposit	External	Payable	-
redeem	External	Can Modify State	-
redeemSingle	External	Can Modify State	-
withdraw	External	Can Modify State	-
withdrawSingle	External	Can Modify State	-
_removeLiquidity	Private	Can Modify State	-
_refund	Private	Can Modify State	-
_zap	Private	Can Modify State	-
_swap	Private	Can Modify State	-

AmbientVaultHelper			
_addLiquidity	Private	Can Modify State	-
_transferFromSender	Private	Can Modify State	-
_transferTo	Private	Can Modify State	-
_normalizeTokenInput	Private	-	-
_getSwapFee	Private	-	-
_getPoolQuoteTokenAmount	Private	-	-
_getPoolBaseTokenAmount	Private	-	-
_safeConvertUint256ToUint128	Private	-	-
_safeSubUint256AndInt256	Private	-	-
<Receive Ether>	External	Payable	-
<Fallback>	External	Payable	-

AmbientVault			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	Vault

CrocLpErc20			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	ERC20
depositCrocLiq	Public	Can Modify State	nonReentrant onlyCrocSwapDex
withdrawCrocLiq	Public	Can Modify State	nonReentrant onlyCrocSwapDex

4.3 Vulnerability Summary

[N1] [Medium] Potential Sandwich Attack Risk

Category: Design Logic Audit

Content

The `_swap` function of the `AmbientVaultHelper` contract will perform the token swap operation through the `deposit` and `single swap level` functions. But the `minOut` parameter is 1, which will not do slippage checking.

There is a risk of being attacked by sandwiches.

Code location:

helper/AmbientVaultHelper.sol#367-404

```
function _swap(PoolInfo memory poolInfo, bool isBuy, bool inBaseQty, uint128 qty,
uint128 limitPrice)
    private
    returns (bytes memory)
{
    if (poolInfo.baseToken == NATIVE_ETH && isBuy == true) {
        return crocSwapDex.userCmd{value: qty}(
            uint16(1),
            abi.encode(
                poolInfo.baseToken,
                poolInfo.quoteToken,
                ...
                limitPrice,
                uint128(1),
                uint8(0)
            )
        );
    } else {
        return crocSwapDex.userCmd(
            uint16(1),
            abi.encode(
                poolInfo.baseToken,
                poolInfo.quoteToken,
                ...
                limitPrice,
                uint128(1),
                uint8(0)
            )
        );
    }
}
```

Solution

It is recommended to perform a slippage check when performing a swap operation.

Status

Fixed

[N2] [Medium] Repeated assignment of the same input

Category: Design Logic Audit

Content

In the deposit function, users can pass in a TokenInput struct to deposit a specified token. However, there is a limit on the length of inputs in the function, which requires it to be less than 3. If the user passes in two tokens of the same address in the struct, the value entered in the structure will first be obtained through the `_transferFromSender` loop and transferred to the contract. When the token is a native token, the same `msg.value` will be assigned to `inputs[0].amount` and `inputs[1].amount` twice, and then it will be assigned to the parameter `normalizedInputs[1]` in the `_normalizeTokenInput` function, and the value assigned to `normalizedInputs[0]` is 0. If the token passed in is not a native token, the `_transferFromSender` function will transfer it to the contract twice through the `safeTransferFrom` function, and the `quoteTokenAmount` or `baseTokenAmount` obtained by `_normalizeTokenInput` is the amount value passed in the last loop and the judgment in the loop can pass. That is, `normalizedInputs[0]` and `normalizedInputs[1]` are the amount of the last loop, and 0 respectively. This will cause the parameter to overwrite, and the user will transfer tokens one more time, resulting in capital loss.

Code location:

helper/AmbientVaultHelper.sol# 57-124, 457-489

```
function deposit(TokenInput[] calldata inputs, LimitPrice calldata limitPrice,
address vault, address receiver)
    external
    payable
    returns (uint256)
{
    require(inputs.length < 3, "TOO_MANY_INPUT_TOKENS");

    // quote token: x, base token: y, native ETH will always be the base token
    PoolInfo memory poolInfo = PoolInfo({
```

```

        ...
    });

    for (uint8 i = 0; i < inputs.length; i++) {
        _transferFromSender(inputs[i].token, inputs[i].amount);
    }
    TokenInput[] memory normalizedInputs = _normalizeTokenInput(vault, inputs);
    ...
}

function _transferFromSender(address token, uint256 amount) private {
    if (token == NATIVE_ETH) {
        require(msg.value == amount, "ETH_AMOUNT_MISMATCH");
    } else {
        IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
    }
}

function _normalizeTokenInput(address vault, TokenInput[] memory inputs)
    private
    view
    returns (TokenInput[] memory)
{
    TokenInput[] memory normalizedInputs = new TokenInput[](2);
    uint256 quoteTokenAmount = 0;
    uint256 baseTokenAmount = 0;

    for (uint8 i = 0; i < inputs.length; i++) {
        if (inputs[i].token == IVault(vault).quoteToken()) {
            quoteTokenAmount = inputs[i].amount;
        } else if (inputs[i].token == IVault(vault).baseToken()) {
            baseTokenAmount = inputs[i].amount;
        } else {
            revert();
        }
    }

    normalizedInputs[0] = TokenInput({token: IVault(vault).quoteToken(), amount:
quoteTokenAmount});
    normalizedInputs[1] = TokenInput({token: IVault(vault).baseToken(), amount:
baseTokenAmount});

    return normalizedInputs;
}

```

Solution

It is recommended to check that the two contract addresses of the passed TokenInput are different.

Status

Fixed

[N3] [Low] Receive can lock users' native tokens**Category: Others****Content**

There is a receive function in the AmbientVaultHelper contract to receive native tokens from the crocSwapDex contract. However, the receive function can lock users' native tokens when users transfer the native token in these contracts by mistake.

Code location:

helper/AmbientVaultHelper.sol#530

```
receive() external payable {}
```

Solution

It's recommended to add addresses that can send the native tokens in the contract in the require check logic in the receive() function.

Status

Acknowledged

[N4] [Suggestion] Reentrancy reminder**Category: Reentrancy Vulnerability****Content**

In the _transferTo function of the AmbientVaultHelper contract, it will transfer or refund the native tokens to the receiver. if the receiver is a malicious contract with a malicious fallback function, it can make the receiver reentry into the AmbientVaultHelper contract.

Code location:

helper/AmbientVaultHelper.sol#459-466

```
function _transferTo(address receiver, address token, uint256 amount) private {  
    if (token == NATIVE_ETH) {
```

```
(bool sent,) = receiver.call{value: amount}("");
require(sent, "FAILED_TO_SEND_ETHER");
} else {
    IERC20(token).safeTransfer(receiver, amount);
}
}
```

Solution

It's recommended to add the nonReentrant lock to prevent the unexpected callback reentrancy.

Status

Fixed

[N5] [Suggestion] Low-level call reminder

Category: Others

Content

In the AmbientVaultHelper contract, the contract uses low-level calls and does not limit the amount of gas used to transfer native tokens.

Code location:

src/helper/AmbientVaultHelper.sol#461

```
(bool sent,) = receiver.call{value: amount}("");
```

Solution

When using low-level calls, it is recommended to limit the amount of gas used.

Status

Fixed

[N6] [Suggestion] Missing the 0 address Check

Category: Others

Content

In the AmbientVaultHelper, the constructor doesn't check whether the input addresses are 0 addresses.

Code location:

helper/AmbientVaultHelper.sol#54-57

```

constructor(ICrocSwapDex _crocSwapDex, ICrocQuery _crocQuery) {
    crocSwapDex = _crocSwapDex;
    crocQuery = _crocQuery;
}

```

Solution

It's recommended to add the check whether the addresses are 0 addresses.

Status

Fixed

[N7] [Suggestion] Token compatibility reminder

Category: Design Logic Audit

Content

In the AmbientVaultHelper contract, users can approve tokens to deposit input tokens by safeTransferFrom and safeTransfer functions, the token amount will be directly recorded, and it will also use the token amount to process the swap operation. If the tokens are deflationary tokens, the actual amount of tokens received will be less than the amount recorded by the amount parameter.

Code location:

helper/AmbientVaultHelper.sol#451-457, 464

```

function _transferFromSender(address token, uint256 amount) private {
    if (token == NATIVE_ETH) {
        require(msg.value == amount, "ETH_AMOUNT_MISMATCH");
    } else {
        IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
    }
}

```

Solution

It is recommended to record the difference before and after the user's transfer as the actual amount or not use the deflationary tokens in this protocol.

Status

Acknowledged; After communicating with the project team, they stated that they will not use deflationary tokens.

[N8] [Suggestion] External call reminder

Category: Unsafe External Call Audit

Content

In the AmbientVaultHelper contract, the external call contracts crocSwapDex and crocQuery are the non-audit scope and play a key role in contract calls, and are closely related to user funds. The project team and users need to pay attention to these external risks when calling these functions.

Code location:

helper/AmbientVaultHelper.sol#277, 372, 388, 415, 432

Solution

It is recommended to clarify if external call contracts are credible and check the validity of the incoming resolver address, data, and business logic.

Status

Acknowledged

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002407150001	SlowMist Security Team	2024.07.11 - 2024.07.15	Low Risk

Summary conclusion: The SlowMist security team use a manual and SlowMist team's analysis tool to audit the project, during the audit work we found 2 medium risks, 1 low risk, 3 suggestions and 2 information. And 2 medium risks and 3 suggestions were confirmed and fixed; All other findings were acknowledged. The code was not deployed to the mainnet.

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



Official Website
www.slowmist.com



E-mail
team@slowmist.com



Twitter
[@SlowMist_Team](https://twitter.com/SlowMist_Team)



Github
<https://github.com/slowmist>